
Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation

Yoshua Bengio, Nicholas Léonard and Aaron Courville
Département d'informatique et recherche opérationnelle
Université de Montréal

Abstract

Stochastic neurons and hard non-linearities can be useful for a number of reasons in deep learning models, but in many cases they pose a challenging problem: how to estimate the gradient of a loss function with respect to the input of such stochastic or non-smooth neurons? I.e., can we “back-propagate” through these stochastic neurons? We examine this question, existing approaches, and compare four families of solutions, applicable in different settings. One of them is the minimum variance unbiased gradient estimator for stochastic binary neurons (a special case of the REINFORCE algorithm). A second approach, introduced here, decomposes the operation of a binary stochastic neuron into a stochastic binary part and a smooth differentiable part, which approximates the expected effect of the pure stochastic binary neuron to first order. A third approach involves the injection of additive or multiplicative noise in a computational graph that is otherwise differentiable. A fourth approach heuristically copies the gradient with respect to the stochastic output directly as an estimator of the gradient with respect to the sigmoid argument (we call this the straight-through estimator). To explore a context where these estimators are useful, we consider a small-scale version of *conditional computation*, where sparse stochastic units form a distributed representation of gates that can turn off in combinatorially many ways large chunks of the computation performed in the rest of the neural network. In this case, it is important that the gating units produce an actual 0 most of the time. The resulting sparsity can be potentially be exploited to greatly reduce the computational cost of large deep networks for which conditional computation would be useful.

1 Introduction and Background

Many learning algorithms and in particular those based on neural networks or deep learning rely on gradient-based learning. To compute exact gradients, it is better if the relationship between parameters and the training objective is continuous and generally smooth. If it is only constant by parts, i.e., mostly flat, then gradient-based learning is impractical. This was what motivated the move from neural networks based on so-called formal neurons, with a hard threshold output, to neural networks whose units are based on a sigmoidal non-linearity, and the well-known back-propagation algorithm to compute the gradients (Rumelhart *et al.*, 1986).

We call the *computational graph* or *flow graph* the graph that relates inputs and parameters to outputs and training criterion. Although it had been taken for granted by most researchers that smoothness of this graph was a necessary condition for exact gradient-based training methods to work well, recent successes of deep networks with rectifiers and other “non-smooth” non-linearities (Glorot *et al.*, 2011; Krizhevsky *et al.*, 2012a; Goodfellow *et al.*, 2013) clearly question that belief: see Section 2 for a deeper discussion.

In principle, even if there are hard decisions (such as the threshold function typically found in formal neurons) in the computational graph, it is possible to obtain *estimated gradients* by introducing *perturbations* in the system and observing the effects. Although finite-difference approximations of

the gradient appear hopelessly inefficient (because independently perturbing each of N parameters to estimate its gradient would be N times more expensive than ordinary back-propagation), another option is to introduce *random perturbations*, and this idea has been pushed far (and experimented on neural networks for control) by Spall (1992) with the Simultaneous Perturbation Stochastic Approximation (SPSA) algorithm.

As discussed here (Section 2), non-smooth non-linearities and stochastic perturbations can be combined to obtain reasonably low-variance estimators of the gradient, and a good example of that success is with the recent advances with *dropout* (Hinton *et al.*, 2012; Krizhevsky *et al.*, 2012b; Goodfellow *et al.*, 2013). The idea is to multiply the output of a non-linear unit by independent binomial noise. This noise injection is useful as a regularizer and it does slow down training a bit, but not apparently by a lot (maybe 2-fold), which is very encouraging. The symmetry-breaking and induced sparsity may also compensate for the extra variance and possibly help to reduce ill-conditioning, as hypothesized by Bengio (2013).

However, it is appealing to consider noise whose amplitude can be modulated by the signals computed in the computational graph, such as with *stochastic binary neurons*, which output a 1 or a 0 according to a sigmoid probability. Short of computing an average over an exponential number of configurations, it would seem that computing the exact gradient (with respect to the average of the loss over all possible binary samplings of all the stochastic neurons in the neural network) is impossible in such neural networks. The question is whether good estimators (which might have bias and variance) or similar alternatives can be computed and yield effective training. We discuss and compare here four reasonable solutions to this problem, present theoretical results about them, and small-scale experiments to validate that training can be effective, in the context where one wants to use such stochastic units to gate computation. The motivation, described further below, is to exploit such sparse stochastic gating units for *conditional computation*, i.e., avoiding to visit every parameter for every example, thus allowing to train potentially much larger models for the same computational cost.

1.1 More Motivations and Conditional Computation

One motivation for studying stochastic neurons is that stochastic behavior may be a required ingredient in *modeling biological neurons*. The apparent noise in neuronal spike trains could come from an actual noise source or simply from the hard to reproduce changes in the set of input spikes entering a neuron’s dendrites. Until this question is resolved by biological observations, it is interesting to study how such noise – which has motivated the Boltzmann machine (Hinton *et al.*, 1984) – may impact computation and learning in neural networks.

Stochastic neurons with binary outputs are also interesting because they can easily give rise to *sparse representations* (that have many zeros), a form of regularization that has been used in many representation learning algorithms (Bengio *et al.*, 2013). Sparsity of the representation corresponds to the prior that, for a given input scene, most of the explanatory factors are irrelevant (and that would be represented by many zeros in the representation).

As argued by Bengio (2013), sparse representations may be a useful ingredient of *conditional computation*, by which only a small subset of the model parameters are “activated” (and need to be visited) for any particular example, thereby greatly reducing the number of computations needed per example. Sparse gating units may be trained to select which part of the model actually need to be computed for a given example.

Binary representations are also useful as keys for a hash table, as in the semantic hashing algorithm (Salakhutdinov and Hinton, 2009). Trainable stochastic neurons would also be useful inside recurrent networks to take hard stochastic decisions about temporal events at different time scales. This would be useful to train multi-scale temporal hierarchies¹ such that back-propagated gradients could quickly flow through the slower time scales. Multi-scale temporal hierarchies for recurrent nets have already been proposed and involved exponential moving averages of different time constants (El Hahi and Bengio, 1996), where each unit is still updated after each time step. Instead, identifying key events at a high level of abstraction would allow these high-level units to only be updated when needed (asynchronously), creating a fast track (short path) for gradient propagation through time.

¹Daan Wierstra, personal communication

1.2 Prior Work

The idea of having stochastic neuron models is of course very old, with one of the major family of algorithms relying on such neurons being the Boltzmann machine (Hinton *et al.*, 1984). Another biologically motivated proposal for synaptic strength learning was proposed by Fiete and Seung (2006). It is based on small zero-mean *i.i.d.* perturbations applied at each stochastic neuron potential (prior to a non-linearity) and a Taylor expansion of the expected reward as a function of these variations. Fiete and Seung (2006) end up proposing a gradient estimator that looks like a *correlation between the reward and the perturbation*, very similar to that presented in Section 3, and which is a special case of the REINFORCE (Williams, 1992) algorithm. However, unlike REINFORCE, their estimator is only unbiased in the limit of small perturbations.

Gradient estimators based on stochastic perturbations have long been shown to be much more efficient than standard finite-difference approximations (Spall, 1992). Consider N quantities u_i to be adjusted in order to minimize an expected loss $L(u)$. A finite difference approximation is based on measuring separately the effect of changing each one of the parameters, e.g., through $\frac{L(u) - L(u - \epsilon e_i)}{\epsilon}$, or even better, through $\frac{L(u + \epsilon e_i) - L(u - \epsilon e_i)}{2\epsilon}$, where $e_i = (0, 0, \dots, 1, 0, 0, \dots, 0)$ where the 1 is at position i . With N quantities (and typically $O(N)$ computations to calculate $L(u)$), the computational cost of the gradient estimator is $O(N^2)$. Instead, a perturbation-based estimator such as found in Simultaneous Perturbation Stochastic Approximation (SPSA) (Spall, 1992) chooses a random perturbation vector z (e.g., isotropic Gaussian noise of variance σ^2) and estimates the gradient of the expected loss with respect to u_i through $\frac{L(u+z) - L(u-z)}{2z_i}$. So long as the perturbation does not put too much probability around 0, this estimator is as efficient as the finite-difference estimator but requires $O(N)$ less computation. However, like the algorithm proposed by Fiete and Seung (2006) this estimator becomes unbiased only as the perturbations go towards 0. When we want to consider all-or-none perturbations (like a neuron sending a spike or not), it is not clear if these assumptions are appropriate. An advantage of the approaches proposed here is that they do not require that the perturbations be small to be valid.

2 Non-Smooth Stochastic Neurons

One way to achieve gradient-based learning in networks of stochastic neurons is to build an architecture in which noise is injected so that gradients can sometimes flow into the neuron and can then adjust it (and its predecessors in the computational graph) appropriately.

In general, we can consider the output h_i of a stochastic neuron as the application of a deterministic function that depends on some noise source z_i and on some differentiable transformation a_i of its inputs (typically, a vector containing the outputs of other neurons) and internal parameters (typically the bias and incoming weights of the neuron):

$$h_i = f(a_i, z_i) \quad (1)$$

So long as $f(a_i, z_i)$ in the above equation has a non-zero gradient with respect to a_i , gradient-based learning (with back-propagation to compute gradients) can proceed. In this paper, we consider the usual affine transformation $a_i = b_i + \sum_j W_{ij} x_{ij}$, where x_i is the vector of inputs into neuron i .

For example, if the noise z_i is added or multiplied somewhere in the computation of h_i , gradients can be computed as usual. Dropout noise (Hinton *et al.*, 2012) and masking noise (in denoising auto-encoders (Vincent *et al.*, 2008)) is multiplied (just after the neuron non-linearity), while in semantic hashing (Salakhutdinov and Hinton, 2009) noise is added (just before the non-linearity). For example, that noise can be binomial (dropout, masking noise) or Gaussian (semantic hashing).

However, if we want h_i to be binary (like in stochastic binary neurons), then f will have derivatives that are 0 almost everywhere (and infinite at the threshold), so that gradients can never flow.

There is an intermediate option that we put forward here: choose f so that it has two main kinds of behavior, with zero derivatives in some regions, and with significantly non-zero derivatives in other regions. We call these two states of the neuron respectively the insensitive state and the sensitive state.

2.1 Noisy Rectifier

A special case is when the insensitive state corresponds to $h_i = 0$ and we have sparsely activated neurons. The prototypical example of that situation is the rectifier unit (Nair and Hinton, 2010;